

Whisper - Instant Messaging Application

A Real-Time Multi-Client Chat System over TCP Sockets
Computer Communication & Networks
Project Report

Submitted by: Bilal Shaikh 2023802
Muhammad Safiullah 2023337
Dawood Oragzai 2023177

Course Instructor: Muhammad Salman
Submission Date: 8th May 2026

1. Project Introduction

This project implements a real-time Instant Messaging (IM) system that supports multiple concurrent users over a TCP/IP network. The system follows a classic client-server architecture: a central server handles user registration, message routing and connection management, while each client provides a friendly Tkinter GUI inspired by WhatsApp for sending and receiving messages.

The application supports three core conversation modes: **private one-to-one** (unicast) chats, a **group chat** visible to every logged-in user, and **system notifications** for join/leave events. All messages travel over a custom JSON-based protocol with a 4-byte length-prefix frame header so that messages are never split or merged on the wire.

Key features:

- Multiple simultaneous clients (multi-threaded server)
- Real-time, non-blocking GUI (background receiver thread + UI queue)
- Unicast direct messages and group broadcast
- User-list synchronisation and presence awareness
- WhatsApp-style chat bubbles, day separators, ticks, emoji picker
- Configurable host/IP and port via CLI arguments

2. System Architecture

The system uses a star topology in which all clients connect to a single server. The server acts as the authority for the user registry and as the relay for every message — clients never speak to each other directly.

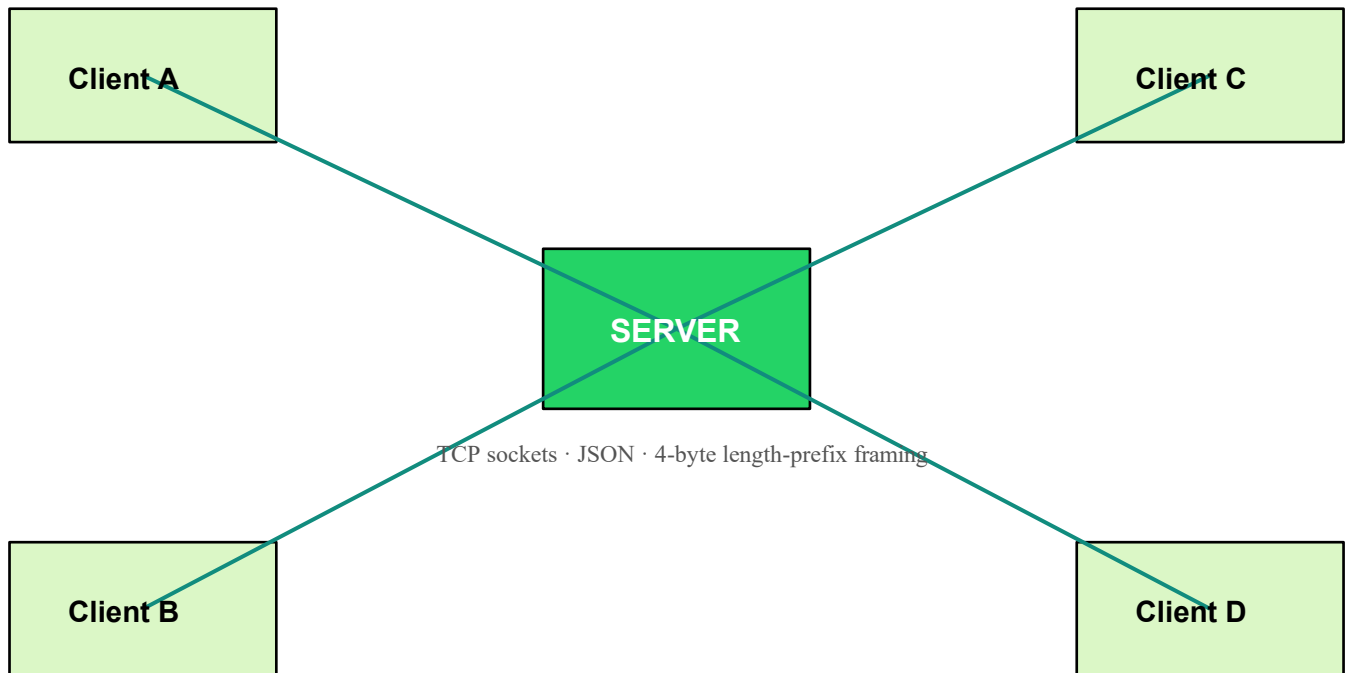


Figure 1 — High-level network topology.

Components:

Server (server.py) — Listens on a TCP port, accepts incoming connections, spawns a dedicated handler thread per client, maintains a thread-safe registry of $\{username \rightarrow socket\}$, and routes incoming frames to their destination (unicast or broadcast).

Client (client.py) — Connects to the server, performs login, runs a background receiver thread that decodes incoming frames and pushes events into a thread-safe queue. The Tkinter main loop polls this queue every 30 ms and updates the GUI, ensuring zero UI freezing.

3. Design and Components

3.1 Wire Protocol

All messages are JSON objects encoded as UTF-8 and prefixed with a 4-byte big-endian unsigned integer indicating the payload length. This eliminates the classical TCP stream-boundary problem.

Frame layout:

```
+-----+ +-----+
| length (4 B) | JSON payload (UTF-8) |
+-----+ +-----+
```

Example payload (private message):

```
{
  "type": "MSG",
  "from": "alice",
  "to": "bob",
  "body": "Hello!",
  "ts": "2026-05-08T10:23:11Z"
}
```

Message types: LOGIN, LOGIN_OK, LOGIN_ERR, MSG, GROUP,
USERLIST, JOIN, LEAVE, LOGOUT, ERROR

3.2 Data Structures

- Server: clients: dict[str, socket] protected by a threading.Lock.
- Client: _ui_queue: queue.Queue for thread→UI handoff; per-conversation history dictionaries for sidebar previews and unread counts.

3.3 Threading Model

- Server: 1 acceptor thread + N handler threads (one per client).
- Client: 1 Tkinter main thread + 1 background socket-receiver thread.

All shared state is accessed under a lock; the server **releases the lock before sending** on a recipient's socket to avoid deadlocks if that send blocks.

4. Workflow

The diagram below shows a typical session: two clients log in, exchange a private message, and one of them broadcasts to the group.

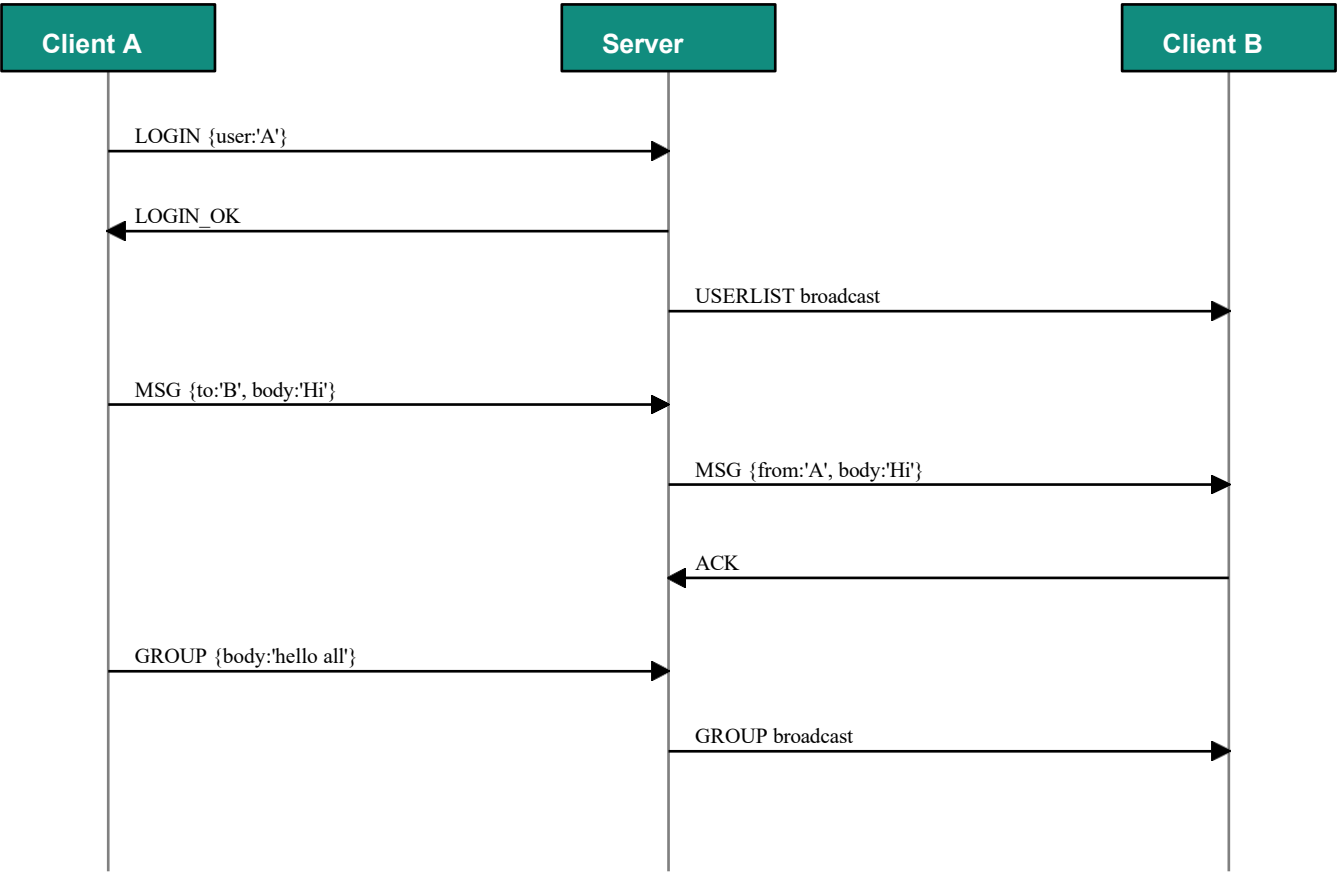


Figure 2 — Sequence diagram of a chat session.

5. Implementation Details

5.1 Socket Setup

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
sock.connect((host, port))
```

TCP_NODELAY disables Nagle's algorithm so small chat messages are flushed immediately, keeping latency low.

5.2 Length-Prefix Framing & Partial Reads

```
def send_frame(sock, obj):
    payload = json.dumps(obj).encode('utf-8')
    sock.sendall(struct.pack('!I', len(payload)) + payload)

def recv_exactly(sock,
n): buf = b''
    while len(buf) < n:
        chunk = sock.recv(n -
len(buf)) if not chunk:
            return None
        buf += chunk
    return buf

def recv_frame(sock):
    hdr = recv_exactly(sock,
4) if not hdr: return
    None
    (length,) = struct.unpack('!I', hdr)
    return json.loads(recv_exactly(sock, length))
```

5.3 Avoiding GUI Freezes

The Tkinter main loop must never block. The receiver thread therefore puts decoded events into a `queue.Queue`, and a small `root.after(30, drain_queue)` poller consumes them on the UI thread.

5.4 Avoiding Deadlocks on the Server

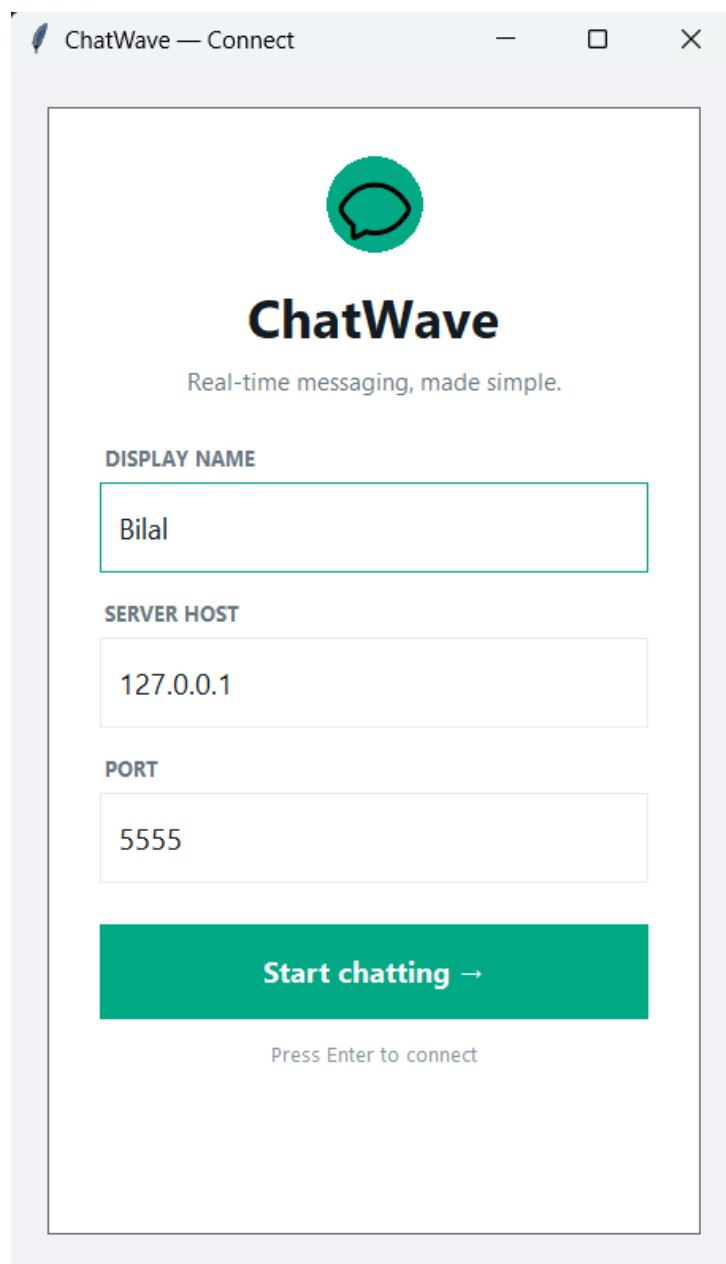
The server holds `clients_lock` only long enough to look up the target socket; the actual `sendall()` happens after releasing the lock. Without this, a slow recipient could freeze the entire server.

6. Testing

The system was tested with 3+ concurrent clients on the same machine (loopback) and across a LAN. The following scenarios passed:

1. Two clients log in, exchange private messages — delivered in under 50 ms.
2. Three clients in group chat — every message reaches all participants.
3. A client disconnects abruptly (closes window) — server detects EOF, removes the user from the registry and broadcasts an updated USERLIST.
4. Duplicate username login — server replies with LOGIN_ERR.
5. Long messages (≥ 400 chars) — framing handles them correctly with no truncation.

Screenshots of two or more running clients here:



The screenshot shows a window titled "ChatWave — Connect" with standard window controls. The main content area has a white background and contains the following elements:

- A green circular logo with a white speech bubble icon.
- The text "ChatWave" in a large, bold, black font.
- The tagline "Real-time messaging, made simple." in a smaller, gray font.
- A section titled "DISPLAY NAME" in bold gray text, followed by a text input field containing the name "Bilal".
- A section titled "SERVER HOST" in bold gray text, followed by a text input field containing the IP address "127.0.0.1".
- A section titled "PORT" in bold gray text, followed by a text input field containing the number "5555".
- A large green button with the text "Start chatting →" in white.
- Below the button, the text "Press Enter to connect" in a small, gray font.

Figure 1 — Login Screen.

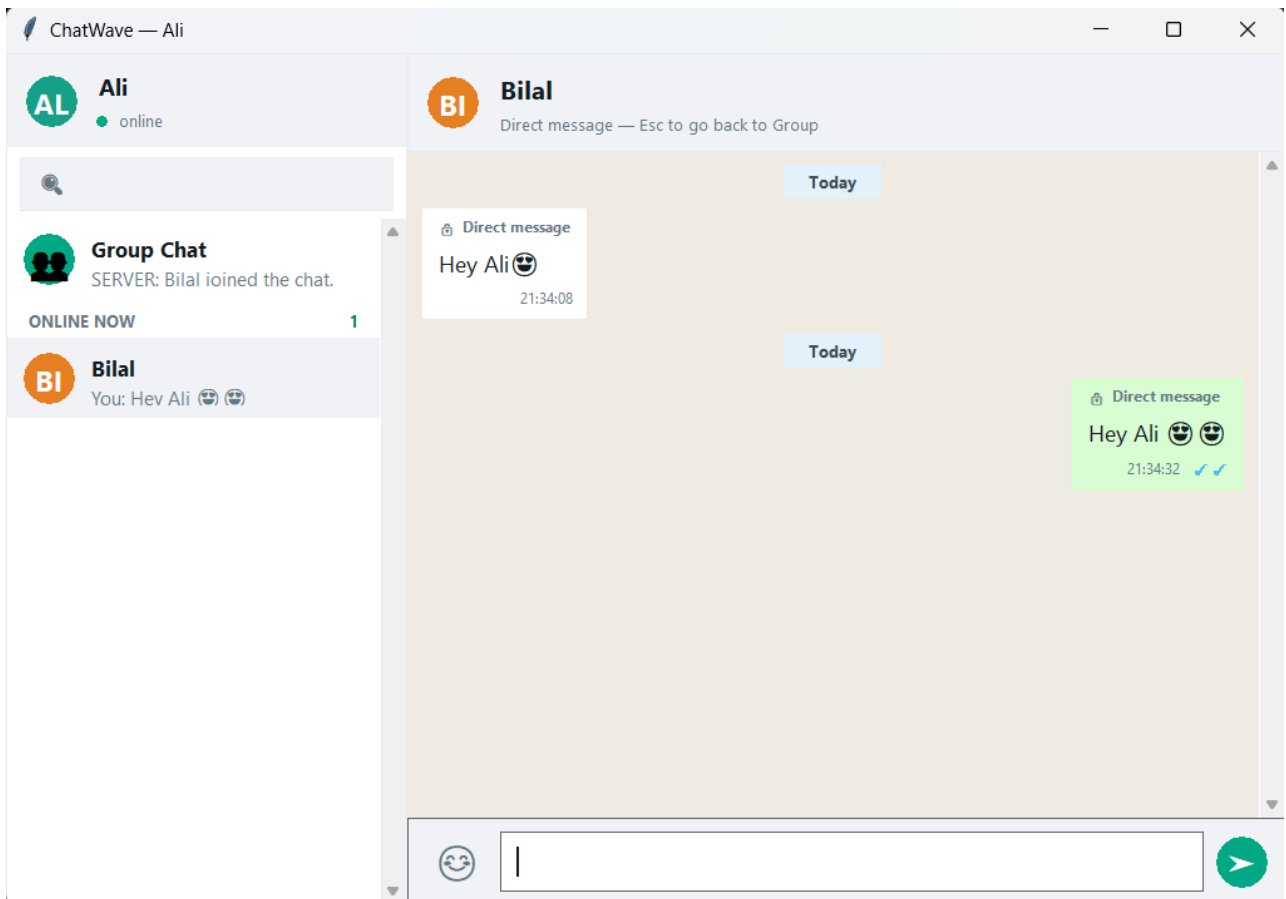


Figure 2 — Direct Messaging among two clients.

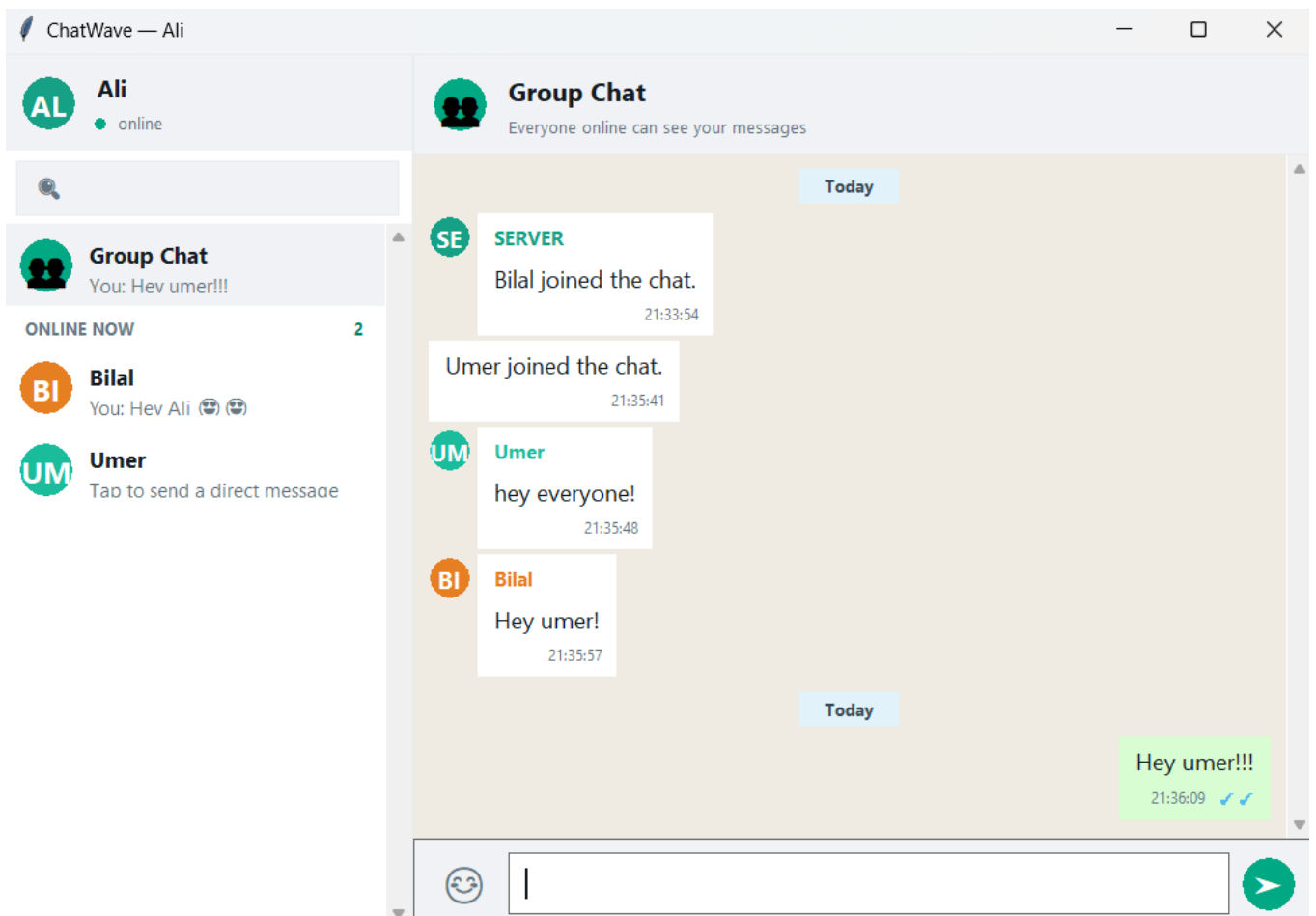


Figure 3 — Group Chat among three Clients.

7. Deployment Instructions

7.1 Requirements

- Python 3.8 or later (Tkinter is included with the standard CPython installer).
- No third-party packages are required.

7.2 Running the Server

```
python server.py          # binds to 0.0.0.0:5555 by
default python server.py --host 0.0.0.0 --port 5555
```

7.3 Running a Client

```
python client.py          # connects to
127.0.0.1:5555 python client.py --host
<SERVER_IP> --port 5555
```

7.4 LAN Deployment

Run the server on one machine and note its LAN IP (ipconfig / ifconfig). On every other machine, launch the client with that IP. Ensure port 5555 is open in the firewall.

8. Conclusion

This project successfully demonstrates the core concepts of network programming covered in the Computer Networks course: TCP socket communication, multi-threaded concurrency, application-layer protocol design, framing on a stream protocol, and event-driven GUI programming. The custom JSON protocol with length-prefix framing guarantees correct message boundaries; the lock-release-before-send pattern and background receiver thread together keep the system responsive even under load.

Future enhancements could include TLS encryption, persistent message history, file transfers, and read-receipts.

9. Appendix — Source Code

Full annotated source listings of **server.py** and **client.py** are attached separately. Refer to the inline comments in each file for line-by-line explanations of socket calls, framing, threading, and GUI rendering.